Viva questions — and how to answer them

Live demo, no slides · AI-Powered Video Summarization for Smart Learning · Batch 1, Team 11

marked ★ = very likely

Before anything else

Answer in the **pipeline** vocabulary. Never say agent, agentic, autonomous, orchestrator, planner, tool calling or self-improving — those cost marks. Say: six stages in a fixed order, three cross-cutting layers, scores reviewed offline by a person.

If you do not know, say **"I am not sure, but I can show you where that is in the code"** — and open the file. Never invent a number.

THE DEMO, IN ORDER — WHAT TO CLICK, WHAT TO SAY

0 · Before they arrive

Server already running. Browser already open on the **finished Unit 5 course run** (5 videos). A second tab on the project folder in the editor, for "show me the code". Never start a fresh run live — it needs network and takes minutes.

1 · Open on the Summary tab

Say: "This is a whole course — a five-video playlist — summarised as one. 6,223 words of transcript, 38 chunks indexed together." Point at the chip row; it proves course mode without a slide.

2 · Click a timestamp in the Timeline

The video jumps to that moment. Say: "Every line is traceable. This is not the model remembering — it is a citation we validated against the transcript." **This is your strongest 30 seconds. Do it early.**

3 · Open the Mind map

Say: "Branches come from across the playlist, not one lecture." Point at two leaves from different videos.

4 · Ask a question in the chat

Use a prepared one that spans lectures — "Give me 3 revision questions" works, it cites V4, V1 and V2. Say: "One answer, three different lectures, each citation clickable."

5 · Ask something the course does not cover

Have one ready. It should refuse. Say: "It says the videos do not cover it instead of inventing an answer. That refusal is the point of the project." **Examiners remember this.**

6 · Open the Evaluation tab and close

Faithfulness 0.688, relevance 0.756, compression 5.9×, 0 injection / safety / bias, 7 weakly grounded sentences flagged. Say: "We flag what we cannot fully ground rather than hide it."

> **If something breaks mid-demo:** say what should have happened, switch to the exported markdown or JSON in `exports/`, and carry on. Do not debug in front of the examiner.

Opening questions and the pipeline

OPENING QUESTIONS

★ What does your project do, in one sentence?

It turns a YouTube video, or a whole course playlist, into a summary, a timeline and a mind map where every line can be traced back to the moment it was said.

★ What problem does it solve that ChatGPT does not?

A general chatbot answers from training memory and cannot point to the minute in the lecture. Ours answers only from this transcript and cites a clickable timestamp, so the student can check it.

Why is this Generative AI and not plain ML?

The summary is new text that appears nowhere in the transcript. Extraction would copy sentences; a decoder-only transformer generates them.

Who is the user?

Students revising recorded lectures and online courses, where a course is five hours of video and the exam is tomorrow.

What is your contribution over an existing summariser?

Three things: grounding to the retrieved transcript only, timestamp validation so citations are real, and course mode so a playlist is handled as one body of material.

PIPELINE — STAGE BY STAGE

★ Walk me through your pipeline.

Ingestion, chunking, embedding, retrieval, generation, output. Layers 7 to 9 — error handling, red teaming, evaluation — apply across all six.

★ How do you get the transcript? Do you use speech-to-text?

No. We read YouTube's own caption track with `youtube-transcript-api`. It already carries start time and duration per segment, which is what makes real timestamps possible. No audio is downloaded, so it is fast and light.

What if a video has no captions?

It is skipped and reported in the run, and the remaining videos continue. One bad video does not sink a thirty-video course.

★ Why do you chunk? Why not send the whole transcript?

Two reasons. The context window is finite — a one-hour lecture may not fit. More importantly, sending everything removes the point of retrieval: we want the model to see only the relevant part, so it cannot wander.

★ Why 1000 characters and 200 overlap?

1000 characters is roughly a paragraph of speech — one idea. The 200-character overlap means an idea explained across a cut appears whole in at least one chunk. Both are in `config.py` and can be changed in one place.

How do your timestamps stay accurate after chunking?

We never cut mid-segment. Chunks are built from whole transcript segments, so each chunk inherits a real start and end time from the caption track.

What cleaning do you do?

Filler words and bracketed noise like `[music]` are removed. Stop words are stripped only from the *search key*, never from the text given to the model — removing them from the context would wreck the writing.

Embedding, retrieval and generation

★ What is an embedding?
A list of numbers representing meaning. Text with similar meaning lands close together in that space, so you can search by idea rather than by keyword.

★ Which embedding model, and how many dimensions?
`all-MiniLM-L6-v2`, 384 dimensions. It is small and runs on a laptop CPU, which matters because we embed every chunk of every video.

Is the embedding model the same as the generation model?
No, and this is the key distinction. MiniLM is *encoder-only* — it reads text and produces a vector. Mistral is *decoder-only* — it writes new text. Two halves of the transformer architecture, two different jobs.

What is your vector database and what similarity do you use?
ChromaDB, with cosine similarity. If Chroma is unavailable we fall back to an in-memory brute-force search so the demo still runs.

★ Explain your retrieval. How many chunks?
We fetch 12 candidates by vector similarity, re-rank them, then keep the top 5. Those 5 are the only text the model ever sees.

★ Why re-rank? Isn't vector search enough?
No. Vector search misses exact terms — ask for "Dijkstra" and it may return something merely about shortest paths. So we blend three signals: **0.75** vector similarity, **0.20** word overlap, **0.05** position in the video. Vector handles paraphrase, lexical handles exact terms, position breaks ties in timeline order.

★ The screen says top-5, but 29 chunks were retrieved. Explain.
Top-5 applies to a *question*. For a *summary* one query would bias toward one part of the video, so we sweep it with four broad coverage probes — main topic, key concepts, worked examples, conclusions — take the union, and force the first and last chunk in so the opening and ending are never missed. Across five videos that totals 29.

★ Which generative model, and why that one?
`open-mistral-nemo`. It is open-weight, cheap, and supports a JSON response mode so the output parses reliably. It is called over plain HTTPS — no SDK — and is swappable from `config.py`.

★ Did you fine-tune or train anything?
No. Fine-tuning bakes facts into weights where you cannot audit them or cite a source. With retrieval the knowledge lives in the transcript, so changing the video changes the answer immediately, with no retraining and no labelled data.

Why force JSON output?
The reply has parts — overview, key points, timestamps, mind map. Parsing those out of free prose is fragile. We request `response_format: json_object` and parse a known schema.

PROMPT ENGINEERING

★ What prompt engineering did you use? Name the techniques.
Four, all in `prompts.py`: **role prompting** (the model is told it is a lecture assistant), a **grounding constraint** (use only the supplied context, never outside knowledge), **structured output** (a JSON schema it must follow), and a **few-shot exemplar** (one worked example showing the shape of a good answer).

Is prompt engineering already in the code, or do you type it in the chat?
It is in the code. Every prompt lives in `prompts.py` so it can be reviewed and changed in one place — the chat box only supplies the user's question.

Show me where the grounding constraint is.
Open `yt_summarizer/prompts.py`. The system prompt states the model may answer only from the numbered context blocks and must say the video does not cover it otherwise.

Course mode, validation and evaluation

MULTI-VIDEO / COURSE MODE

★ How does a playlist work? Do you run the pipeline five times?

Partly. We list the playlist with `yt-dlp` (index only, no media), pull each transcript, and chunk them all into *one shared index*. Each video is then summarised on its own, and those summaries are synthesised into one course guide — map-reduce, because a whole course never fits in a single prompt.

★ Why one shared index rather than one per video?

So a question can search every lecture at once. A topic taught in three lessons should return all three, which separate indexes cannot do.

★ Won't one long video dominate the results?

It would with plain top-k, which is why we cap each video at **two chunks** per answer. We pull a wider candidate set first, then apply the cap, then take the best overall.

How does a citation identify the video?

In a course it becomes `V4 0:00` — video index plus timestamp. For a single video it stays plain `4:32`, so the old behaviour is unchanged.

What is "where is this covered?"

An endpoint that answers which lessons teach a topic, rather than just the single best match.

VALIDATION, SAFETY AND EVALUATION

★ What stops the model inventing a timestamp?

Every timestamp it returns is checked against the chunks we actually retrieved, with a 15-second tolerance, and snapped to the nearest real one. Anything that does not match is discarded. This was a real bug — the model returned 4:35 for a chunk at 4:32, and exact matching threw away every citation.

★ What are the red-team screens?

Four, run on every output: **prompt injection** (transcripts containing "ignore your instructions", detected and neutralised), **content safety**, **bias**, and **grounding**. Our last run was 0, 0, 0, with 7 weakly grounded sentences flagged.

What does "weakly grounded" mean — is it an error?

It means we could not match that sentence's content words back to the retrieved text. It is a warning shown to the user, not a deletion. We would rather flag than quietly hide.

★ How do you evaluate? Explain each metric.

Faithfulness 0.688 — how much of the summary is supported by the retrieved text. **Relevance 0.756** — cosine similarity between the summary and the centre of the retrieved chunks. **Compression 5.9×** — how much shorter the summary is. **ROUGE** — overlap with a reference summary, only when one is supplied.

★ Is 0.688 a good score?

Be honest: it is a *lexical proxy*, not a true entailment check, so we read it as a trend rather than a grade. A paraphrase that is perfectly correct can still score low.

Why is ROUGE not computed on your run?

ROUGE needs a human-written reference summary to compare against. We have none for these lectures, so it is labelled as not computed rather than filled with a fake number.

Do the evaluation scores change anything automatically?

No. They are reported and reviewed by a person offline. Nothing in the system re-tunes itself.

Code, limits and hard questions

CODE AND ENGINEERING

★ Show me the file for stage *n*.

The package maps one-to-one: `stage1_ingest.py` … `stage6_output.py`, then `layer7_errors.py`, `layer8_redteam.py`, `layer9_eval.py`. Settings in `config.py`, prompts in `prompts.py`, the sequence in `pipeline.py`.

What is your tech stack?

Python with FastAPI and Uvicorn for the server, ChromaDB for vectors, sentence-transformers for embeddings, Mistral over HTTPS for generation, and a hand-written HTML/CSS/JS front end — no framework.

Did you test it?

24 tests in `test_pipeline.py`, no pytest needed — `python test_pipeline.py`. One of them specifically pins that a single video still behaves exactly as it did before course mode was added.

Can it run without internet or an API key?

Yes, in offline mode on the bundled sample lecture and three-video sample course. The output is labelled as degraded so nobody mistakes it for a real run.

What happens if the API call fails?

Layer 7 handles it — errors are caught, labelled and surfaced, and where a fallback exists it is marked as a fallback in the export rather than passed off as the real thing.

LIMITATIONS — ANSWER THESE HONESTLY

★ What is the biggest weakness?

We are only as good as the transcript. Auto-captions mis-hear technical terms, and no amount of prompting recovers a word that was never captured.

What would you do with another month?

Multilingual transcripts, quiz and flashcard generation from the timeline, a labelled set so ROUGE becomes meaningful, and shared course libraries for a class.

Does it scale to a 50-video playlist?

It runs, but cost and time scale with video count — roughly one model call per video plus one to synthesise. That is a known limit, not a surprise.

THE HARD ONES

★ Is this an agentic AI system?

No. It is a pipeline. Six stages run in the same fixed order every time. It does not plan, choose tools, act on its own or modify itself. Evaluation output is read by a person afterwards.

So where is the “decision making” then?

In deterministic rules, not autonomy: the re-ranking weights decide which chunks are kept, the validator decides which timestamps survive, the grounding rule decides when to refuse. All fixed, all inspectable.

What stops someone hiding instructions in a transcript to hijack your model?

That is prompt injection, and it is screened in layer 8 — detected and neutralised before the text reaches the model. It is also why the model is told to treat context as data, not instructions.

Can it hallucinate at all?

Yes — we reduce it, we do not eliminate it. That is exactly why we flag weakly grounded sentences and validate timestamps instead of claiming perfect accuracy.

What did each of you build?

Answer by ownership, which is what the sheet assigns: problem framing and the front end; pipeline stages 1 to 4; model, prompts and evaluation; course mode, validation and testing. Agree this between the four of you *before* the demo.

> **Last thing.** Have the app already open on the Unit 5 course run before the examiner arrives. Do not start a fresh run live — it takes time, needs network, and a failure in front of an examiner is a mark you cannot get back.